ChurnGuard Analytics Platform

Predictive Churn Analytics — Technical Consulting Report

Prepared: May 02, 2026 | Prepared by: Muhammad Junaid Baig

Executive Summary

This report documents ChurnGuard Analytics — a full-stack predictive churn intelligence system built for the Customer Success team. The system ingests three SaaS data sources (5,000 accounts, 13,618 users, 22,385 support tickets), engineers 63 features, trains an XGBoost binary classifier, and delivers predictions through an interactive desktop application with a natural-language chat interface, a live risk dashboard, SHAP-based explanations, and a full suite of model diagnostic tools.

A key finding from this engagement is the identification of synthetic data leakage: account_health_score is engineered with a hard 20-point gap (churned accounts score 0–40, active accounts score 60–100) that trivially separates classes, producing AUC = 1.00 on the full-feature model. A second feature, risk_flag, shows signs of post-hoc labelling. Both features were identified through a rigorous correlation-based diagnostics workflow built into the application. After excluding these two features, the debiased model achieves AUC = 0.9992, F1 = 0.9860 — a more honest baseline that reflects genuine behavioural signals. All recommendations in this report are grounded in the debiased model.

1 | Data Analysis

1.1 Dataset Overview

Three CSV sources were provided. A schema-driven data loader (core/data_loader.py) reads all sources from config/schema.yaml, so adding a new Phase 2 source requires only a YAML entry — no code changes.

Source	Rows	Columns	Grain
account_lifecycle_events.csv	5,000	26	One row per account
user_engagement_metrics.csv	13,618	21	One row per user — multiple per account
support_interaction_history.csv	22,385	20	One row per ticket — multiple per account

Target variable: account_status = 'churned'. Class distribution across subscription tiers:

Tier	Accounts	Churned	Churn Rate
Free	1,009	128	12.7%
Starter	1,786	194	10.9%
Professional	1,462	152	10.4%
Enterprise	743	72	9.7%
TOTAL	**5,000**	**546**	**10.9%**

1.2 Data Quality Findings

The following issues were identified through the in-app Diagnostics window (🔍 Diagnose), which runs a point-biserial correlation audit against the churn label for all 63 features and flags zero-overlap distributions. Details are in Section 4.

- **CRITICAL — account_health_score:** Zero overlap between churned (0–40) and active (60–100) distributions. |r| = 0.85. Synthetic gap makes this feature trivially separate classes — see Section 4.
- **HIGH — risk_flag:** |r| = 0.82 with churn label. Set by CS team during quarterly reviews; may reflect known churn intent rather than predict it. See Section 4.
- **MEDIUM — One-sided support signals:** cancellation_requested_count, downgrade_requested_count, competitor_mention_count: |r| range 0.65–0.79. Legitimate signals but over-represented in synthetic data (active accounts never have any such tickets in this dataset).
- **INFO — status_change_date excluded:** Intentionally omitted: computing days_since_status_change = 0 for recently churned accounts would constitute direct label leakage.
- **INFO — Satisfaction rating nulls:** ~5% null rate on satisfaction_rating due to low survey response. Imputed with column median.
- **INFO — EU account dual identifiers:** EU accounts created after January 2023 carry both account_id and account_uuid (GDPR compliance). The data loader handles both join keys transparently.

1.3 Feature Engineering

63 features were engineered from the three source tables. All column references are schema-driven (config/schema.yaml) — renaming or adding a column requires only a config change. The feature_exclusions list in config/model_config.yaml allows any feature to be toggled out of training without code changes.

Category	Count	Key Features
Time-based recency / age	4	account_age_days, days_since_last_activity, days_until_contract_end, post_health_algo era flag
Seat & contract	4	seat_utilization_rate, mrr, seats_purchased, seats_active
Account signals	9	integration_count, account_health_score, risk_flag, has_account_manager, tier/region/billing/provisioning encodings
User engagement aggregates	18	login_count_30d/90d, feature_usage_score, session_duration, active_user_pct, onboarding_completed_rate, certification_earned_rate, etc.
Support aggregates	28	ticket_count, tickets_last_30d/90d, cancellation/downgrade/pause/escalation counts, negative_sentiment_rate, CSAT mean, competitor_mention_count, etc.

2 | Application Overview

2.1 Architecture

The system is built on a multi-agent architecture (agents/orchestrator.py) where a central Orchestrator owns all shared state and routes messages between six specialised agents. This separation means any component can be swapped or extended without affecting others — a deliberate design choice to support Phase 2 adaptability.

Agent	Responsibility
DataPipelineAgent	Schema-driven CSV loading, type coercion, feature engineering. Supports dynamic source addition at runtime.
ModelAgent	XGBoost training, prediction, TreeSHAP explanation generation, and

	model cache management.
AnalyticsAgent	Portfolio summaries, top-risk rankings, MRR-at-risk calculations. Read-only — never modifies state.
ConversationAgent	Pattern-based NL router: parses 9 intent types, dispatches to 12 handler methods, formats responses.
CurveballAgent	Runtime scenario injection for Phase 2 testing — modifies in-memory frames and forces model retrain.

2.2 Application Features

The desktop application (Python / tkinter) exposes four interactive surfaces:

- **Main Dashboard:** Three-tab panel: Overview (portfolio totals, model performance metrics), Charts (probability distribution histogram, feature importance bar chart, per-account SHAP waterfall), and Risk Table (accounts sortable and filterable by High/Low risk tier). All panels refresh automatically after any model retrain.
- **Chat Interface:** Natural-language query box supporting account-level SHAP explanations ("Why is ACC000123 predicted to churn?"), portfolio queries ("Show top 10 at-risk accounts"), MRR-at-risk, tier/region breakdowns, and model performance queries. Clicking any row in the Risk Table auto-fills and sends an explanation query.
- **Model Performance Window (📊 Model):** Four diagnostic charts in a single window: ROC curve with AUC annotation, Precision-Recall curve with Average Precision, confusion matrix at 70% threshold, and overlapping score distribution histogram by actual class (churned vs retained).
- **Diagnostics Window (🔍 Diagnose):** Three-tab leakage audit: a sortable table of all features ranked by |r| with the churn label (highlighting zero-overlap features in red), a horizontal bar chart of correlations with significance thresholds at |r|=0.5 and 0.8, and per-feature distribution grids showing churned vs retained histograms for the top 6 features by correlation.
- **Feature Settings Dialog (⚙ Features):** Modal dialog for toggling feature exclusions (with rationale badges for known leaky features), triggering an in-app retrain without restart, and generating this report. The dashboard refreshes with new metrics as soon as training completes.

3 | Model Development — Full-Feature Baseline

3.1 Algorithm Selection: XGBoost

Algorithm: XGBoost binary classifier (xgboost.XGBClassifier). Justification:

- Tabular data with mixed feature types (continuous, ordinal, boolean) — XGBoost handles all of these natively without requiring one-hot encoding or feature scaling.
- Class imbalance (10.9% churn rate, ~1:8 ratio) — scale_pos_weight is computed automatically as n_negative / n_positive ≈ 8.2, upweighting minority-class errors during training.
- Built-in feature importance and full compatibility with XGBoost's native TreeSHAP (pred_contribs=True), avoiding the shap library entirely and eliminating version incompatibility risks.
- Strong empirical performance on tabular datasets of this size (5,000 rows, 63 features) without requiring feature scaling, interaction engineering, or imputation strategies.
- Fast inference: all 5,000 accounts scored in milliseconds, keeping the dashboard refresh responsive even with live retrains.
- Regularisation parameters (gamma, reg_alpha, reg_lambda) are exposed in config/model_config.yaml to control overfitting on the relatively small dataset.

Known limitation: gradient boosted trees find and fully exploit any feature with near-zero within-class overlap (like account_health_score). This is the correct algorithmic behaviour — the model is doing exactly what it should — but it makes synthetic leakage more visible and impactful than it would be with a regularised linear model. For production deployment, probability calibration (Platt scaling or isotonic regression) is recommended to convert raw scores into well-calibrated likelihoods.

3.2 Training Methodology

- Stratified 80/20 train/test split (random_seed=42) — preserves the 10.9% churn rate in both partitions.
- 5-fold stratified cross-validation on the full dataset for an unbiased AUC estimate independent of the holdout split.
- Class imbalance corrected with scale_pos_weight = n_negative / n_positive (≈ 8.2 for this dataset).
- Hyperparameters (n_estimators=300, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8) set in config/model_config.yaml.
- Model artifacts (XGBClassifier, feature names, metrics) cached to models/churn_model.joblib. SHAP contributions cached to models/shap_values.npy.
- Cache is invalidated and a fresh retrain is triggered whenever the feature exclusion list changes or a CurveballAgent scenario modifies the feature distribution.

3.3 Performance Metrics — Full-Feature Model (63 features)

⚠ **All metrics = 1.00 due to account_health_score leakage identified in Section 4. These figures reflect the synthetic dataset only. Section 5 reports the debiased metrics.**

Metric	Value	Interpretation
ROC-AUC	1.0000	Model correctly ranks churned above healthy 100% of the time — driven by the leaky health score.
PR-AUC	1.0000	Perfect precision-recall tradeoff — artifact of the zero-overlap health score distribution.
Accuracy	1.0000	100% of test accounts correctly classified at the 70% threshold.
Precision	1.0000	Every account flagged as high-risk truly churned — no false positives.
Recall	1.0000	Every churned account was captured — no missed churns.
F1 Score	1.0000	Harmonic mean of precision and recall. Confirms zero errors on this dataset.
CV AUC Mean	1.0000	5-fold CV AUC = 1.00 confirms leakage is structural, not overfitting.
CV AUC Std	0.0000	Zero variance across folds — leakage affects every data split equally.

3.4 Feature Importance — Full-Feature Model (Global SHAP)

TreeSHAP values computed for all 5,000 accounts. Mean absolute SHAP = average contribution to the log-odds prediction across the portfolio. account_health_score dominates by a factor of ~8× over the next feature, confirming it as the sole driver of AUC = 1.00.

Feature	Mean |SHAP|	Business Meaning
account_health_score	5.7267	⚠ Leaky — churned 0–40, active 60–100, zero overlap. Replace in production.
downgrade_requested_count	0.7105	Downgrade requests strongly precede full cancellation.
days_since_last_activity	0.2621	Inactivity > 30 days → 3× higher churn rate.

competitor_mention_count	0.1644	Competitor mentions in tickets signal active evaluation of alternatives.
integration_count	0.1142	Zero integrations → 4× higher churn; shallow product adoption.
feature_usage_score	0.1139	Low feature breadth signals low perceived platform value.
risk_flag	0.1033	⚠️ CS at-risk flag — strong but potentially post-hoc (see Section 4).
account_pause_requested_count	0.0955	Pause requests are a softer but reliable churn precursor.
cancellation_requested_count	0.0800	Direct cancellation intent — any non-zero count is high risk.
avg_session_duration_minutes	0.0141	Shorter sessions may indicate frustration or disengagement.
resolution_time_hours_mean	0.0136	Slow ticket resolution correlates with dissatisfaction.
tier_rank	0.0081	Lower-tier accounts churn at slightly higher rates (12.7% vs 9.7%).

4 | Leakage Investigation & Testing

4.1 Diagnostics Workflow

The AUC = 1.00 result prompted a systematic leakage investigation using the in-app Diagnostics window (🔍 Diagnose). The workflow ran three tests across all 63 features:

- Point-biserial correlation (|r|) between each continuous feature and the binary churn label. Features with |r| > 0.5 are flagged; |r| > 0.8 triggers a CRITICAL warning.
- Zero-overlap check: for each feature, compare max(churned) vs min(active) and max(active) vs min(churned). Any non-overlapping distribution is marked as a potential leakage artifact.
- Per-feature distribution grids: overlapping histograms of churned vs retained accounts for the top 6 features by correlation, to visually confirm the separation.

4.2 Findings

account_health_score (|r| = 0.85, ZERO OVERLAP):

The churned distribution is entirely contained in 0–40 (mean ≈ 20); the active distribution is entirely contained in 60–100 (mean ≈ 80). There is a hard 20-point gap with no accounts between scores of 40 and 60. This is a synthetic data artifact: the data generation algorithm encoded churn status directly into the health score range. A single split on this feature classifies every account correctly, which is why AUC = 1.00 and CV AUC = 1.00 with zero variance across all 5 folds. We also confirmed this by examining the probability distribution histogram — even at 101 bins (1% resolution), the bimodal shape shows two perfectly separated spikes at 0% and 100% with no middle-ground probabilities, which is the hallmark of a leaky feature.

risk_flag (|r| = 0.82, potential post-hoc labelling):

Set by the CS team during quarterly business reviews. The criteria for setting this flag are internal and may incorporate knowledge that the account is already at risk of churning — effectively making it a lagging indicator masquerading as a leading one. In the dataset, risk_flag = True for 100% of churned Professional/Enterprise accounts and 0% of active ones in those tiers, suggesting the flag was applied retrospectively. Time-gating the flag (only use values set ≥ 30 days before the evaluation date) would be the production-safe alternative to outright exclusion.

5 | Debiased Model — After Feature Exclusion

5.1 Features Excluded and Justification

- **account_health_score:** Zero overlap between churned (0–40) and active (60–100) distributions. |r| = 0.85. The synthetic data generation algorithm encoded churn status directly into the health score range — any tree split on this feature produces perfect class separation. Removing it forces the model to learn from genuine behavioural signals.
- **risk_flag:** Point-biserial |r| = 0.82. Set by the CS team during quarterly reviews; criteria are internal and may incorporate knowledge that the account is already intending to churn. In production this should be time-gated (only use flags set ≥ 30 days before evaluation) rather than excluded; for this analysis, exclusion gives the most conservative baseline.

5.2 Performance Comparison — Original vs Debiased

The debiased model was trained after running rebuild_features with both features excluded, then force_retrain=True. The same 80/20 stratified holdout was used for both models. The drop in AUC reflects the removal of the leaky signal, not any degradation in genuine predictive ability.

Metric	Original	Debiased	Interpretation
ROC-AUC	1.0000	**0.9992**	Ranking accuracy — still near-perfect without leaky features
PR-AUC	1.0000	0.9951	Area under precision-recall curve
Accuracy	1.0000	0.9970	% of test accounts classified correctly at 70% threshold
Precision	1.0000	1.0000	Of accounts flagged high-risk, fraction that truly churn
Recall	1.0000	0.9725	Fraction of actual churners captured
F1 Score	1.0000	0.9860	Harmonic mean of precision and recall
CV AUC	1.0000	0.9985	Cross-validated AUC — 5 stratified folds
CV AUC ±	0.0000	0.0013	Variation across folds (lower = more stable)

5.3 Feature Importance — Debiased Model (Global SHAP)

With the leaky features removed, SHAP rankings shift toward engagement and intent signals that CS teams can actually act on. The top features are now downgrade requests, pause requests, and inactivity — all of which are observable and intervenable.

Feature	Mean |SHAP|	Business Meaning
downgrade_requested_count	2.7621	Now the dominant signal — customers who request downgrades almost always churn.
account_pause_requested_count	0.8377	Pause requests rise sharply when account is considering exit.
days_since_last_activity	0.8084	Engagement recency is the strongest time-based churn predictor.
competitor_mention_count	0.6781	Competitor mentions in support tickets indicate evaluation behaviour.
integration_count	0.5312	Accounts with zero integrations churn 4× more often.
cancellation_requested_count	0.5188	Any cancellation ticket is a near-certain churn signal.
days_until_contract_end	0.3663	Accounts close to renewal are at heightened

		risk.
feature_usage_score	0.3658	Breadth of feature adoption is the most improvable leading indicator.
ticket_count	0.3639	High ticket volume without resolution → escalating frustration.
seat_utilization_rate	0.2527	Under-utilised seats signal underperceived value.
reopened_count_mean	0.2446	Reopened tickets indicate unresolved problems building over time.
avg_session_duration_minutes	0.2181	Short sessions signal disengagement or inability to find value.

6 | Business Recommendations

6.1 Actionable Churn Drivers (CS Playbook)

The following signals are both predictive in the debiased model and actionable by the CS team — they can be observed before churn and intervened on:

Signal	Recommended Action
Downgrade / cancellation ticket	Route to CS within 24 hrs. In the dataset 0% of active accounts file these — any non-zero count is near-certain churn intent.
Competitor mention in ticket	Escalate to senior CS for competitive retention conversation within 48 hrs.
Days since last activity > 30	Automated health-check email at 30 days; CS phone call at 60 days.
Account pause requested	Treat as an early-exit signal; offer a pause option with a re-engagement incentive.
Integration count = 0	Proactive onboarding call within 14 days of sign-up. Zero integrations → 4× higher churn.
Feature usage score < 40	Schedule a product training session. Breadth-of-adoption is the most improvable leading indicator.
Active user % < 30%	Audit inactive users; offer seat right-sizing to demonstrate value and reduce waste.
Seat utilization < 50%	Review with account champion — low utilisation is a common precursor to downsizing.

6.2 Risk Thresholds

Thresholds are configurable in config/model_config.yaml without code changes. Based on the debiased model's precision-recall curve:

Tier	Threshold	Recommended Action
High Risk	≥ 70%	Immediate CS outreach this week. Prioritise by MRR. In the dataset this captures 543 accounts representing essentially all churned MRR.
Elevated	20–69%	Proactive health-check email; add to CS watchlist for next cycle.
Low Risk	< 20%	Standard monitoring; no immediate action required.

Portfolio summary (debiased model): 5,000 accounts, 543 high-risk (≥70%), 4,455 low-risk (<20%), average churn probability 11.1%, max 99.99%. The bimodal distribution confirms the model assigns decisive scores — most accounts sit clearly in one tier or the other with very few borderline cases.

6.3 Production Deployment Considerations

- Replace account_health_score with a non-leaky composite built from engagement metrics (login frequency, feature adoption, seat utilization) — preserves the business concept without the synthetic gap.
- Time-gate risk_flag: only use values set ≥ 30 days before the evaluation date to avoid feedback-loop leakage from CS team foreknowledge.
- Add a temporal holdout (train on months 1–10, validate on months 11–12) once time-series data is available. The current stratified random split does not validate temporal generalisation.
- Apply probability calibration (Platt scaling or isotonic regression via sklearn's CalibratedClassifierCV) to convert raw XGBoost scores into well-calibrated likelihoods suitable for business threshold decisions.
- Retrain monthly or when feature distribution drift is detected on key signals (days_since_last_activity, downgrade_requested_count).

6.4 Monitoring Recommendations

- Feature drift: alert if mean(days_since_last_activity) or mean(integration_count) shifts by > 1 standard deviation month-over-month.
- Prediction distribution drift: alert if the fraction of accounts scoring ≥ 70% doubles or halves unexpectedly between scoring runs.
- Outcome feedback: log actual churn events and compare to predicted probabilities monthly; recalibrate thresholds if precision falls below 0.70.
- CS response rate: track what % of high-risk-flagged accounts received outreach within 7 days to measure operational adoption.

7 | Phase 2 Adaptability

7.1 Curveball Framework

The CurveballAgent (agents/curveball_agent.py) provides runtime scenario injection: it modifies in-memory DataFrames, forces a model retrain, then can roll back — all without touching disk or restarting the application. Seven scenarios are implemented and accessible via /curveball <name> in the chat interface or python main.py --curveball <name>:

Scenario	What It Tests
column_rename	Schema drift: a CSV column is renamed. Tests that the schema-driven loader handles remapping via config/schema.yaml without code changes.
new_source	Adds a synthetic NPS data source at runtime. Tests DataPipelineAgent.add_source() and the generic aggregation fallback for unknown source types.
churn_redefine	Expands churn definition to include suspended accounts. Tests the target config in schema.yaml (target.extra_positive_values).
null_injection	Injects 30% nulls into key engagement fields. Tests the median-imputation pipeline end-to-end.
new_tier	Introduces an unseen 'platinum' subscription tier. Tests ordinal encoding robustness for out-of-vocabulary categories.
drop_column	Removes integration_count from the feature matrix. Tests that the model gracefully handles missing columns via reindex fill_value=0.
scale_test	Doubles the dataset to ~10,000 accounts. Tests pipeline throughput and UI responsiveness under 2× load.

7.2 Key Architectural Decisions Supporting Phase 2

- Schema-driven design: all data source paths, column names, and the target definition live in config/schema.yaml — a new source or renamed column is a one-line config change.
- Feature exclusions in config: the feature_exclusions list in config/model_config.yaml lets any feature be toggled out of training at runtime without code changes.
- DataPipelineAgent.rebuild_features(): re-runs feature engineering on in-memory frames without disk re-read, so CurveballAgent modifications to state.frames are preserved.
- Agent message passing: new agents can be registered at any time; the Orchestrator routes messages by name, so Phase 2 additions don't require changes to existing agents.
- Generic aggregation fallback (FeatureEngineer._agg_generic): any numeric/boolean source added dynamically gets automatically aggregated by mean/true-rate without custom code.